

# Część II

## Grupa 8 (TP):

Wprowadzenie: 22 kwietnia 2026

Termin wysłania Części II: 6 maja 2026

(z ewentualną możliwością przesunięcia do 13.05.2026, do przedyskutowania na zajęciach)

Termin oddania: 6 maja 2026

## Grupy 3,6 (TN):

Wprowadzenie: 15 kwietnia 2026

Termin wysłania Części II: 13 maja 2026

Termin oddania: 13 maja 2026

## Cel projektu

Na poprzedniej liście przekonaliście się, jak trudne i żmudne jest ręczne wymyślanie reguł decyzyjnych (if/else) dla dużych zbiorów danych. Celem tej listy jest zrozumienie, jak działają klasyczne algorytmy uczenia maszynowego (Machine Learning), które automatyzują ten proces.

Zamiast jednak polegać wyłącznie na gotowych bibliotekach ("czarnych skrzynkach"), **Waszym zadaniem będzie samodzielne zaimplementowanie podstawowych algorytmów (od zera, używając np. biblioteki NumPy) i porównanie ich wyników z gotowymi rozwiązaniami z biblioteki scikit-learn.** Drugim, równie ważnym celem, jest **interpretacja** wyuczonych modeli. Algorytmy będą trenowane z ograniczeniami (np. płytkie drzewa), abyście mogli dokładnie przeanalizować, na jakiej podstawie podejmują one decyzje.

## Zbiory danych

Możesz kontynuować pracę na zbiorach danych wybranych na Liście 1 (pamiętając o podziale na klasyfikację i regresję) lub wybrać nowe, jeśli poprzednie okazały się zbyt trudne do obróbki. **Wymagania dotyczące podziału na zbiór treningowy i testowy (Data Leakage) obowiązują na każdym etapie tej listy!**

---

## Część implementacyjna

**Rekomendacja:** Zaimplementowane przez Ciebie algorytmy powinny mieć postać klas w języku Python, posiadających metody `fit(X, y)` (do treningu) oraz `predict(X)` (do wnioskowania), co ułatwi ich bezpośrednie porównanie z klasami ze scikit-learn.

### **Zakres prac na ocenę 3.0 (Drzewa Decyzyjne)**

- Stwórz model decyzyjny, wykorzystując do tego drzewo decyzyjne (DecisionTreeClassifier ze scikit-learn). Model musi być wytrenowany na zbiorze treningowym. Model powinien na wejściu przyjmować wszystkie istotne dane (kolumny) i zwracać wynik decyzji (target).
- Pamiętaj o właściwym przygotowaniu danych w ramach preprocessingu.
- Przelicz metryki (np. Accuracy, F1 Score, Precision-Recall) na zbiorze testowym. Pamiętaj, aby Twoje metryki umożliwiały stwierdzenie jakości modelu na niezbalansowanym zbiorze danych.

### **Zakres prac na ocenę 3.5 (Interpretacja Drzewa i Ważność Cech)**

- **Wizualizacja Reguł:** Wyciągnij strukturę wytrenowanego na ocenę 3.0 Drzewa Decyzyjnego. W tym celu, ustaw niewielkie ograniczenie głębokości (np. `max_depth=5`, jedno drzewo decyzyjne). Prześledź od korzenia do liści, **jakie konkretnie zasady (if/else) wygenerował algorytm**. Wydrukuj je w konsoli w czytelnej formie (np. `if wiek < 30 and dochod >= 5000 -> klasa 1`). Porównaj to z systemem reguł, który wymyśliłeś na Liście 1.
- **Zysk Informacyjny a Ważność Cech:** Zaimplementuj od zera (np. w NumPy) funkcje obliczające Entropię oraz Zysk Informacyjny (Information Gain). Użyj ich, aby zweryfikować wyniki zwracane przez atrybut `.feature_importances_` z Twojego modelu. Wybierz jedną z najważniejszych cech oraz jedną z najmniej ważnych, a następnie dokonaj "ręcznego" podziału zbioru treningowego na podstawie każdej z nich (wykorzystując próg podziału odczytany z wizualizacji drzewa). Oblicz i porównaj Zysk Informacyjny z obu podziałów. Udowodnij swoimi wyliczeniami, że cecha o najwyższym feature importance faktycznie zapewnia znacznie wyższy spadek zanieczyszczenia (Information Gain) niż cecha o niskiej ważności.  
Zobacz: <https://homes.cs.washington.edu/~shapiro/EE596/notes/InfoGain.pdf>

### **Zakres prac na ocenę 4.0 (Regresja Liniowa)**

- **Własna implementacja:** Zaimplementuj od zera algorytm **Regresji Liniowej (Linear Regression)** dla problemu regresji. Wymagane jest przygotowanie dwóch niezależnych metod trenowania modelu (znajdowania optymalnych wag):
  1. **Rozwiązanie analityczne** z wykorzystaniem macierzy pseudoodwrotnej (Metoda Najmniejszych Kwadratów)
  2. **Rozwiązanie iteracyjne** z wykorzystaniem algorytmu Spadku Gradientu (Gradient Descent).
- **Porównanie:** Wytrenuj oba swoje warianty oraz model LinearRegression ze scikit-learn na tym samym zbiorze treningowym. Porównaj błąd MSE/MAE na zbiorze testowym.
- Zwizualizuj wyniki na zbiorze testowym.

**Weryfikacja wag:** Wypisz wektory wyuczonych wag z obu Twoich algorytmów i porównaj je z atrybutem `.coef_` z modelu scikit-learn. **Wszystkie trzy podejścia powinny zwrócić dokładnie te same wyniki** (z dokładnością do błędu numerycznego i pod warunkiem odpowiedniego doboru parametrów uczenia/liczby epok dla algorytmu Spadku Gradientu).

### **Zakres prac na ocenę 4.5 (Interpretacja Regresji i Skalowanie)**

- **Analiza wag:** Przeanalizuj wagi uzyskane przez modele Regresji Liniowej. Co w kontekście Twojego zbioru danych oznacza, że waga przy konkretnej cesze wynosi np. 3.5 lub -1.2? Która cecha ma największy wpływ na wynik?
- **Skalowanie:** Użyj narzędzia `StandardScaler` (lub własnej implementacji standaryzacji), aby przeskalować cechy zbiorów treningowego i testowego. Wytrenuj Regresję Liniową ponownie na przeskalowanych danych. Zobacz, jak zmieniły się wagi. Wyjaśnij, dlaczego bez uprzedniego skalowania bezpośrednie porównywanie wag między różnymi cechami (np. wiek w latach vs dochód w dolarach) może prowadzić do fałszywych wniosków o "ważności" danej cechy.

### **Zakres prac na ocenę 5.0 (Bias-Variance i Problem Czarnego Łabędzia)**

- **Wymuszanie skrajnych stanów (Overfitting i Underfitting):** Wykorzystaj modele drzew decyzyjnych oraz regresji liniowej (z transformacją `PolynomialFeatures`). Skonfiguruj hiperparametry tak, aby celowo uzyskać dwa stany. Dla underfittingu stwórz model zbyt prosty (np. `max_depth=1` dla drzewa lub czysta regresja liniowa dla nieliniowych danych), pokazując wysokie błędy na zbiorze treningowym i testowym. Dla overfittingu stwórz model o ogromnej złożoności (np. drzewo bez limitu głębokości lub regresja wielomianowa stopnia 15+), udowadniając, że błąd treningowy spada niemal do zera, podczas gdy błąd testowy drastycznie rośnie. Zobacz: [https://nustat.github.io/STAT303-3-class-notes/Bias\\_variance\\_code.html](https://nustat.github.io/STAT303-3-class-notes/Bias_variance_code.html) i [https://scikit-learn.org/stable/auto\\_examples/model\\_selection/plot\\_underfitting\\_overfitting.html](https://scikit-learn.org/stable/auto_examples/model_selection/plot_underfitting_overfitting.html)
- **Krzywe Złożoności (Analiza Bias-Variance):** Zbuduj pętlę iteracyjnie zwiększającą złożoność modelu (np. stopień wielomianu od 1 do 15 lub głębokość drzewa od 1 do 20). W każdej iteracji zapisz wyniki dla zbioru uczącego i testowego, a następnie wygeneruj wykres liniowy z dwiema krzywymi (*Train Error vs Test Error*). Zaznacz na nim punkt optymalny (*sweet spot*), w którym model najlepiej generalizuje wiedzę, zanim zacznie się przeuczać.
- **Problem "Czarnego Łabędzia" (Błąd Ekstrapolacji):** Przetestuj zachowanie swojego najbardziej przeuczonego modelu w zderzeniu z nieprzewidzianą anomalią. Dla regresji wielomianowej wysokiego stopnia poproś o predykcję dla punktu znajdującego się nieco poza zakresem danych treningowych (np. wartość  $X$  o 20% większa niż maksimum w zbiorze). Pokaż i porównaj z modelem optymalnym, jak funkcja przeuczona "eksploduje", zwracając absurdalnie wielkie wartości predykcji, udowadniając całkowity brak zdolności do radzenia sobie z "Czarnymi Łabędziami".

---

## **Cześć raportowa**

Raport z tej części projektu musi być nastawiony nie tylko na pokazanie samych tabel z metrykami (choć one są kluczowe w celu weryfikacji Twojej implementacji), ale przede wszystkim na **zrozumienie modeli matematycznych**.

**W raporcie odpowiedz na następujące pytania / zawrzyj następujące elementy:**

1. **Weryfikacja implementacji:** Przedstaw tabelę porównującą metryki (Accuracy dla klasyfikacji, MSE/MAE dla regresji) pomiędzy Twoimi klasami autorskimi a klasami ze scikit-learn. Skomentuj ewentualne rozbieżności.
2. **Analiza Drzewa Decyzyjnego:** Wklej zasady decyzyjne z drzewa decyzyjnego (z etapu 3.5). Czy warunki wybrane przez algorytm mają sens dziedzinowy? Które cechy okazały się najważniejsze (Feature Importance) i czy pokrywa się to z Twoją intuicją z Listy 1?
3. **Znaczenie Wag w Regresji (Ocena 4.0 - 4.5):** Potwierdź, że obie zaimplementowane metody (analityczna i gradientowa) zbiegły się do tych samych wag co algorytm ze scikit-learn. Zaprezentuj wagi cech przed i po skalowaniu. Skomentuj ludzkim językiem przynajmniej jedną wyuczoną wagę (np. "Wzrost wieku o 1 rok powoduje średni wzrost kosztów ubezpieczenia o X, przy zachowaniu innych parametrów bez zmian"). Dlaczego skalowanie wpłynęło na możliwości interpretacji tego, która cecha jest "najważniejsza"?
4. **Bias-Variance i Czarny Łabędź (Ocena 5.0):** Przedstaw i skomentuj wygenerowane wykresy krzywych złożoności, jednoznacznie identyfikując na nich strefy wysokiego obciążenia (High Bias – underfitting) oraz wysokiej wariancji (High Variance – overfitting). Wyjaśnij koncepcyjnie, dlaczego wraz ze wzrostem skomplikowania algorytmu i "zapamiętywaniem" szumu ze zbioru uczącego, błąd treningowy stale maleje, podczas gdy błąd testowy po pewnym czasie zaczyna gwałtownie rosnąć. Następnie odnieś się do problemu "Czarnego Łabędzia", opisując na podstawie swojego eksperymentu, dlaczego modele przeuczone (takie jak wielomiany wysokiego stopnia) są ekstremalnie wrażliwe na dane spoza wąskiego zakresu treningowego, co skutkuje katastrofalnymi błędami ekstrapolacji. Na koniec podsumuj krótko, w jaki sposób uodparniamy modele na takie anomalie, wymieniając po jednej technice ograniczania złożoności (regularyzacji) dla drzew decyzyjnych oraz modeli regresyjnych.

(Przejsście przez te algorytmy krok po kroku pozwoli Wam zrozumieć ich fundamentalne różnice – Drzewa tworzą ostre, geometryczne podziały przestrzeni w oparciu o entropię, Regresja szuka optymalnej płaszczyzny przecinającej punkty za pomocą algorytmów optymalizacyjnych).